CSE233 Week 2: Variables, Primitive Types, and Arithmetic

In this week, we'll discuss primitive data types, variables and operators. We'll also discuss reading input from the console.

Primitive Types

Below is a table showing the primitive data types available in C++. There are a few others, but they aren't relevant to us in this class.

Name	Description	Minimum Value	Maximum Value
`int`	Whole numbers, including negatives. Uses 4 bytes.	-2147483648	2147483647
`long`	Whole numbers, including negatives. Uses 8 bytes.	-9223372036854775808	9223372036854775807
`float`	Decimal numbers, including negatives. Uses 4 bytes.	-3.40282e+38	3.40282e+38

Name	Description	Minimum Value	Maximum Value
`double`	Decimal numbers, including negatives. Uses 8 bytes.	-1.79769e+308	1.79769e+308
`unsigned`	Whole, positive numbers. Uses 4 bytes.	0	4294967295
`unsigned long`	Whole, positive numbers. Uses 8 bytes.	0	18446744073709551615
`char`	Single text character	N/A	N/A
`bool`	Stores `true` or `false`	N/A	N/A

C++ does not have a string data type by default. `std::string` is provided by the `string` library. To use it, you'll need to have the statement `#include <string>` at the top of the file with `#include <iostream>`. We'll discuss strings in-depth in week 10.

Declarations

In Visual Basic, declarations look like this:

```
Dim x As Integer
Dim y As Double
Dim z As String
```

The equivalent declarations in C++ would be:

```
int x;
double y;
std::string z;
```

C++ does not have a keyword like VB (`Dim`) for a declaration. To declare a variable in C++, you put the data type, followed by the name of the variable. Variable names in C++ consist of letters, numbers, and underscores. Variable names in C++ are typically `camelCase`, although `snake_case` is also used. `camelCase` starts with a lowercase word, and all successive words start with an uppercase letter. `snake_case` uses all lowercase, and the words are separated by underscores. It doesn't matter which you use, but be consistent. Pick a style and stick with it. Variables can contain numbers, but cannot start with them.

A variable can be made constant by adding the `const` specifier before the name of the type. If you make a variable `const`, you must immediately assign it a value with the equals sign:

```
const double pi = 3.14;
const double e = 2.71828;
```

Examples

```
int x;
long y = 1000000;
float pi = 3.14;
double z;
unsigned u = 20;
unsigned long large_number = 18446744073709551615;
char c = 'x'; // character are in single quotes
bool b = true; // bool can only be true or false
```

Arithmetic Operators

Below are the common math, comparison, and boolean operators.

Numeric

Below is a table describing the arithmetic operations that can be performed on float, double, int, and long.

Operator	Description	Example
+	Adds two numbers	`x = 1 + 4 // x is 5`
-	Subtracts two numbers	`x = 5.5 - 2 // x is 3.5`
*	Multiplies two numbers	`x = 3 * 8 // x is 24`
/	Division of two integers (int result, always rounds down)	`x = 3 / 4 // x is 0`
/	Division of at least one floating point number	`x = 3.0 / 4 // x is 0.75`
%	Remainder of dividing two numbers	`x = 5 % 3 // x is 2`

Note that if both operands are integers, integer division will always be performed. If you want a floating point result, at least one of the numbers being divided needs to be a float or double.
There are also bit shift operations that can be performed on numbers, but those are not needed in this class.

Comparison and Boolean

The following operators are used for boolean expressions:

Operator	Description	Example
<	Returns true if the left operand is less than the right	`1 < 2 // True`
>	Returns true if the left operand is greater than the right	`1 > 2 // False`
<=	Returns true if the left operand is less than or equal	`1 <= 2 // True`

Operator	Description	Example
	to the right	
>=	Returns true if the left operand is greater than or equal to the right	`1 >= 2 // False`
==	Returns true if the left operand is the same as the right	`1 == 2 // False`
!=	Returns true if the left operand is different from the right	`1 != 2 // True`
&&	Returns true if both operands are true	`1 < 2 && 1 == 2 // False since 1 == 2 is False`
!	Negates a Boolean expression	`!(1 == 2) // True, 1 == 2 is False, not False is True`

String Concatenation

Strings can be combined using the `+` sign. This is called concatenation. Example: `std::string s = "hello " + "world";`

Reading Input

Just like `std::cout` is used to write to the console, `std::cin` is used to read input from the console. Instead of the insertion operator `<<`, the extraction operator `>>` is used to read input. Below is an example program that prompts the user to enter an integer and prints out the result of the input multiplied by two.

```
// need for std::cout and std::cin
#include <iostream>

int main() {
```

```
    // declare variable for input
    int x;
    // print out prompt
    std::cout << "Enter an integer: ";
    // read number from console
    // std::cin can read any primitive type
    // or string from the console
    std::cin >> x;
    // print out the input number times two
    std::cout << "Result is " << x * 2 << std::endl;

    return 0;
}
```

`std::cin` can read any data primitive data type from the terminal. It can also read a `std::string`. One thing to note is that `std::cin` only reads until it reaches a whitespace character. Consider the program below:

```
// for std::cout and std::cin
#include <iostream>
// for std::string
#include <string>

int main() {
    // declare string for input
    std::string name;
    // prompt
    std::cout << "Enter your name: ";
    // read input
    std::cin >> name;
    // echo input
    std::cout << "You entered: " << name << std::endl;

    return 0;
}
```

Here is some example output:

```
Enter your name: John
You entered: John
```

This is fine. But what if a first and last name were entered?

```
Enter your name: John Doe
You entered: John
```

Only the first word that was entered is read into `name`. The second word remains in the input buffer. We'll talk about buffers later in the course, but for now remember that you can't read strings separated by spaces from the console.

Examples

Here are a few example programs with comments explaining them.

Reading an Address

```
/*
Reads an address from a user
*/
#include <iostream>
#include <string>

// using snake_case for this program
int main() {
    // variable declarations
    int number;
    std::string street_name;
    std::string street_type;
    std::string city;
    std::string state;
    int zip_code;

    // read in the components of the address
    std::cout << "Enter street number: ";
    std::cin >> number;
    std::cout << "Enter street name: ";
    std::cin >> street_name;
    std::cout << "Enter street type (Rd, St, Ave, etc.): ";
    std::cin >> street_type;
    std::cout << "Enter the city: ";
    std::cin >> city;
    std::cout << "Enter the state: ";
    std::cin >> state;
    std::cout << "Enter the ZIP Code: ";
    std::cin >> zip_code;

    // print the address formatted like:
    /*
    123 Main St
```

```
    Somewhere, OH, 44444
    */
    std::cout << number << " "
        << street_name << " "
        << street_type << std::endl;
    std::cout << city << ", "
        << state << ", "
        << zip_code << std::endl;

    return 0;
}
```

Calculate Tax

```
/*
Reads in an item price and computes the tax and total
*/
#include <iostream>

// using camelCase for this program
int main() {
    // declare variable for price
    double itemPrice;
    // declare const for tax rate
    const double taxRate = 0.07;

    // read item price from user
    std::cout << "Enter item price: $";
    std::cin >> itemPrice;

    // calculate tax
    double tax = itemPrice * taxRate;
    // calculate total
    double total = itemPrice + tax;

    // print subtotal, tax, and total
    std::cout << "Subtitle: " << itemPrice << std::endl;
    std::cout << "Tax:      " << tax << std::endl;
    std::cout << "Total:    " << total << std::endl;

    return 0;
}
```

Employee Pay

```
/*
Reads employee name and hours
Computes their pay
*/
```

```cpp
#include <iostream>
#include <string>

int main() {
    // declare variables
    const double hourly_rate = 12.75;
    std::string name;
    const double hours_worked;

    // read name
    std::cout << "Enter name: ";
    std::cin >> name;

     // read hours worked
     std::cout << "Enter hours worked: ";
     std::cin >> hours_worked;

    // calculate pay
    double pay = hourly_rate * hours_worked;

    // print pay
    std::cout << name << " is getting paid $"
        << pay << std::endl;

    return 0;
}
```

Conclusion

This week, we learned about the primitive data types, like `int`, `double`, `char`, and `bool`. We also introduced `std::string` for holding text. We then looked at the syntax for a declaration statement. To declare a variable in C++, you put the data type, followed by the name. Optionally, you can put an equal sign and assign a value to the variable. Constants are declared using the `const` specifier before the type name. We looked then looked at the math, comparison, and boolean operators. Finally, we discussed reading user input from the console.
